LAB TASK - 6

Name - Dhruv Rai
Roll No. - 2023UCS1700

1.To segment objects from an image using thresholding techniques and analyze them using connected component labeling and contour detection.

```
import cv2
import numpy as np
import sys

def nothing(x):
    pass

def color_labels(labels):
    if labels.max() == 0:
        return np.zeros((labels.shape[0], labels.shape[1], 3), dtype=np.uint8)

    hue = np.uint8(179 * labels / np.max(labels))
    blank = 255 * np.ones_like(hue)
    labeled_img = cv2.merge([hue, blank, blank])
    labeled_img = cv2.cvtColor(labeled_img, cv2.COLOR_HSV2BGR)
    labeled_img[hue == 0] = 0
    return labeled_img

def main():
    if len(sys.argv) < 2:
        print("Usage: python lab1_segmentation.py <image_path>")
        return

    image_path = sys.argv[1]
    image = cv2.imread(image_path)

    if image is None:
        print("Error: Could not read image.")
        return
```

```python
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    blurred = cv2.GaussianBlur(gray, (5, 5), 0)

    cv2.namedWindow("Controls")
    cv2.createTrackbar("Threshold", "Controls", 127, 255, nothing)
    cv2.createTrackbar("Min Area", "Controls", 100, 5000, nothing)
    cv2.createTrackbar("Mode: 0=Manual, 1=Otsu", "Controls", 1, 1, nothing)

    while True:
        thresh_val = cv2.getTrackbarPos("Threshold", "Controls")
        min_area = cv2.getTrackbarPos("Min Area", "Controls")
        mode = cv2.getTrackbarPos("Mode: 0=Manual, 1=Otsu", "Controls")

        if mode == 0:
            _, binary = cv2.threshold(blurred, thresh_val, 255, cv2.THRESH_BINARY)
        else:
            _, binary = cv2.threshold(
                blurred, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU
            )

        kernel = np.ones((3, 3), np.uint8)
        binary = cv2.morphologyEx(binary, cv2.MORPH_OPEN, kernel, iterations=1)
        binary = cv2.morphologyEx(binary, cv2.MORPH_CLOSE, kernel, iterations=1)

        num_labels, labels, stats, centroids = cv2.connectedComponentsWithStats(
            binary, connectivity=8
        )

        cc_output = image.copy()
        contour_output = image.copy()

        for i in range(1, num_labels):
            x = stats[i, cv2.CC_STAT_LEFT]
            y = stats[i, cv2.CC_STAT_TOP]
            w = stats[i, cv2.CC_STAT_WIDTH]
            h = stats[i, cv2.CC_STAT_HEIGHT]
```

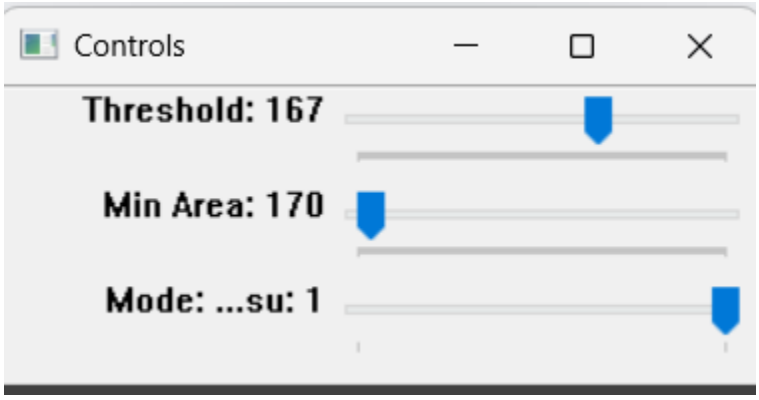
```
        cx, cy = centroids[i]

        if area >= min_area:
            cv2.rectangle(cc_output, (x, y), (x + w, y + h), (0, 255, 0), 2)
            cv2.circle(cc_output, (int(cx), int(cy)), 4, (0, 0, 255), -1)
            cv2.putText(
                cc_output,
                f"ID:{i} A:{area}",
                (x, max(20, y - 8)),
                cv2.FONT_HERSHEY_SIMPLEX,
                0.5,
                (255, 0, 0),
                1
            )

    contours, _ = cv2.findContours(binary, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

    object_count = 0
    for cnt in contours:
        area = cv2.contourArea(cnt)
        if area >= min_area:
            object_count += 1
            x, y, w, h = cv2.boundingRect(cnt)
            cv2.drawContours(contour_output, [cnt], -1, (0, 255, 0), 2)
            cv2.rectangle(contour_output, (x, y), (x + w, y + h), (255, 0, 0), 2)
            cv2.putText(
                contour_output,
                f"A:{int(area)}",
                (x, max(20, y - 8)),
                cv2.FONT_HERSHEY_SIMPLEX,
                0.5,
                (0, 0, 255),
                1
            )

    colored_labels = color_labels(labels)
```

```
        info_panel = np.zeros((120, image.shape[1], 3), dtype=np.uint8)
        cv2.putText(info_panel, f"Connected Components (excluding background): {num_labels - 1}", (10, 30),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 255), 2)
        cv2.putText(info_panel, f"Contours after area filtering: {object_count}", (10, 65),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 255), 2)
        cv2.putText(info_panel, "Press S to save outputs, ESC to exit", (10, 100),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.6, (200, 200, 200), 1)

        cv2.imshow("Original", image)
        cv2.imshow("Gray", gray)
        cv2.imshow("Binary", binary)
        cv2.imshow("Connected Components Labels", colored_labels)
        cv2.imshow("Connected Components Analysis", cc_output)
        cv2.imshow("Contour Detection", contour_output)
        cv2.imshow("Info", info_panel)

        key = cv2.waitKey(1) & 0xFF

        if key == 27:
            break
        elif key == ord('s'):
            cv2.imwrite("output_binary.png", binary)
            cv2.imwrite("output_labels.png", colored_labels)
            cv2.imwrite("output_cc_analysis.png", cc_output)
            cv2.imwrite("output_contours.png", contour_output)
            print("Outputs saved.")

    cv2.destroyAllWindows()

if __name__ == "__main__":
    main()
```

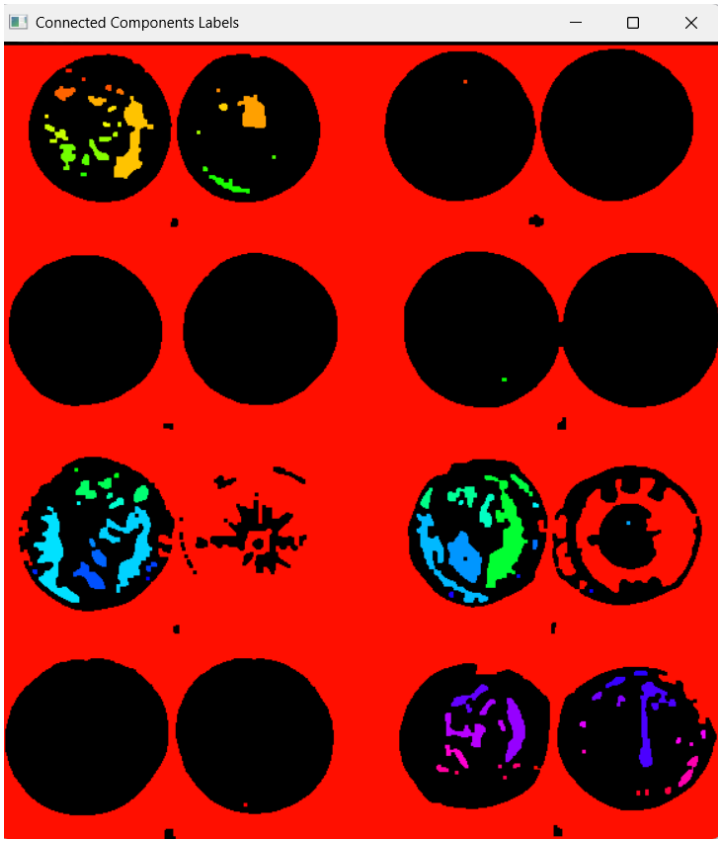
INPUT IMAGE -

OUTPUT -

Controls

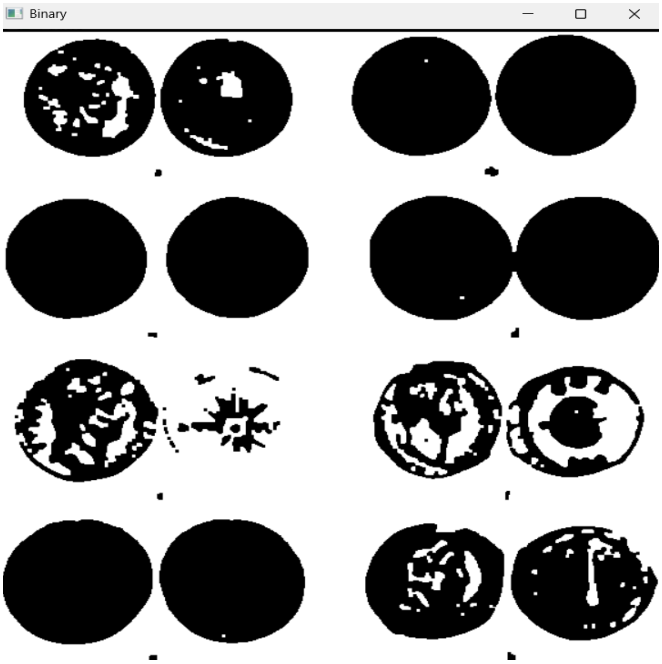


Connected Component Labels



Connected Components Analysis

Binary



Contour Detection

Information

2. To detect key feature points in images and match them between two images using feature descriptors.

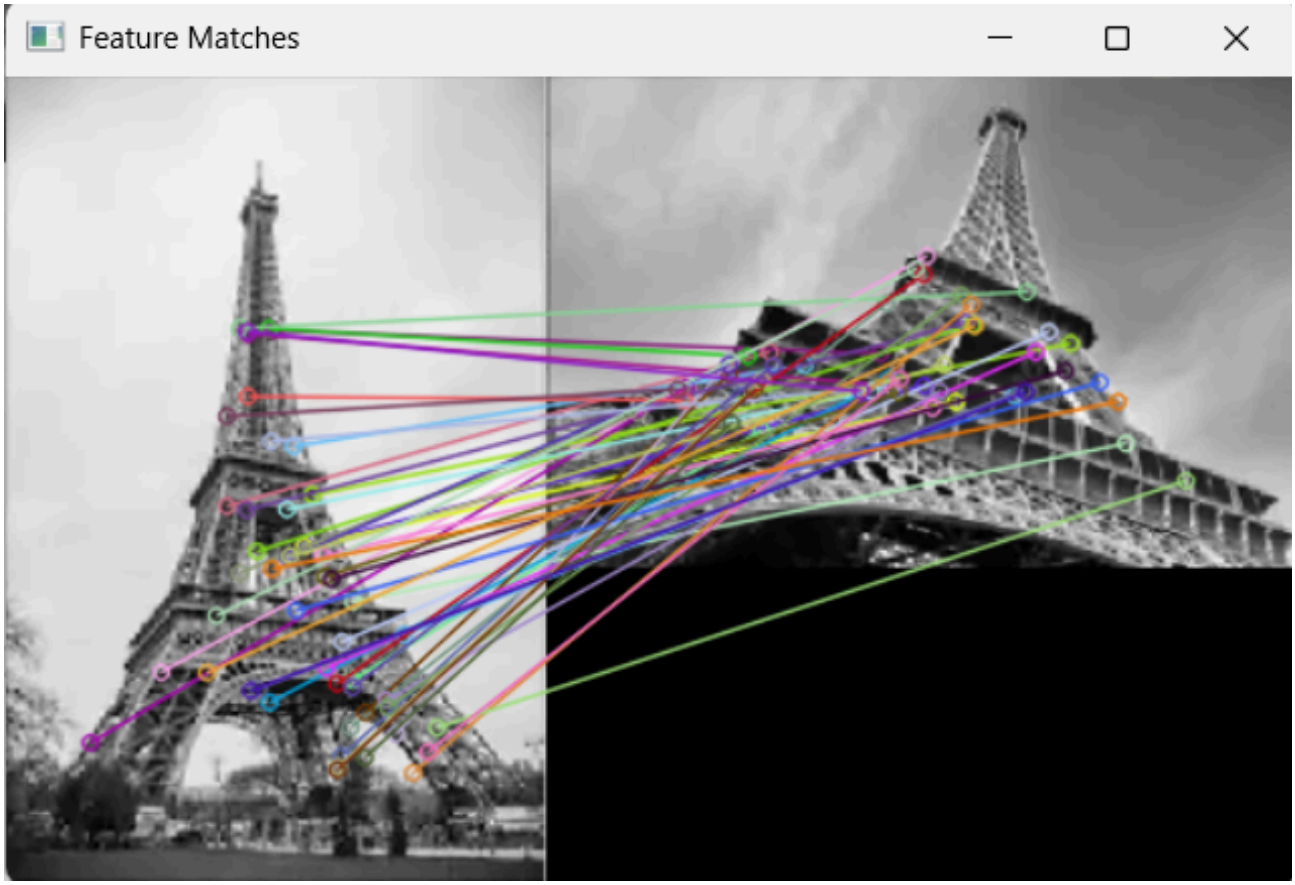
```python
import cv2
import numpy as np
import sys

def main():
    if len(sys.argv) < 3:
        print("Usage: python lab2_feature_matching.py <image1_path> <image2_path>")
        return

    img1_path = sys.argv[1]
    img2_path = sys.argv[2]

    img1 = cv2.imread(img1_path)
    img2 = cv2.imread(img2_path)

    if img1 is None or img2 is None:
        print("Error: Could not read one or both images.")
        return

    gray1 = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)
    gray2 = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)

    orb = cv2.ORB_create(nfeatures=1000)

    kp1, des1 = orb.detectAndCompute(gray1, None)
    kp2, des2 = orb.detectAndCompute(gray2, None)

    if des1 is None or des2 is None:
        print("Not enough features found in one of the images.")
        return

    bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)
    matches = bf.match(des1, des2)

    matches = sorted(matches, key=lambda x: x.distance)

    top_n = min(50, len(matches))
```

```python
    good_matches = matches[:top_n]

    img1_keypoints = cv2.drawKeypoints(
        img1, kp1, None, color=(0, 255, 0),
        flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
    )

    img2_keypoints = cv2.drawKeypoints(
        img2, kp2, None, color=(0, 255, 0),
        flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
    )

    matched_img = cv2.drawMatches(
        img1, kp1, img2, kp2, good_matches, None,
        flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
    )

    print(f"Number of keypoints in image 1: {len(kp1)}")
    print(f"Number of keypoints in image 2: {len(kp2)}")
    print(f"Total matches found: {len(matches)}")
    print(f"Top matches drawn: {top_n}")

    cv2.imshow("Image 1 Keypoints", img1_keypoints)
    cv2.imshow("Image 2 Keypoints", img2_keypoints)
    cv2.imshow("Feature Matches", matched_img)

    key = cv2.waitKey(0) & 0xFF

    if key == ord('s'):
        cv2.imwrite("image1_keypoints.png", img1_keypoints)
        cv2.imwrite("image2_keypoints.png", img2_keypoints)
        cv2.imwrite("feature_matches.png", matched_img)
        print("Outputs saved.")

    cv2.destroyAllWindows()
```

```python
if __name__ == "__main__":
    main()
```

INPUT IMAGE -

IMAGE 1

IMAGE 2

OUTPUT -